

Лабораторное задание № 6

Что такое «сложный запрос» в SQL

- **Простой запрос**
 - Затрагивает обычно **одну таблицу**.
 - Минимум условий, без вложенных подзапросов и JOIN.
- **Сложный запрос**
 - Может использовать **несколько таблиц**.
 - Содержит **объединения таблиц (JOIN)**, **вложенные подзапросы**, **агрегатные функции + GROUP BY / HAVING**, сложные условия в WHERE и ORDER BY.

По Гавриловой: сложные запросы строятся на базе обычного SELECT, но добавляются:

- многотабличные запросы (разные типы JOIN),
- агрегатные функции с группировкой,
- вложенные и коррелированные подзапросы.

3. Базовая структура запроса SELECT

Стандартный шаблон сложного запроса:

```
SELECT  список_столбцов_или_выражений
FROM    таблица_или_несколько_таблиц
[JOIN   другие_таблицы ON условия_соединения]
[WHERE  условия_фильтрации_строк]
[GROUP BY столбцы_группировки]
[HAVING условия_фильтрации_групп]
[ORDER BY столбцы [ASC|DESC]]
[LIMIT  число_строк]  -- или аналог (TOP и т.п.)
```

Логический порядок выполнения (важно для понимания сложных запросов):

1. FROM (+ все JOIN)

2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

Усложнение через условия фильтрации (WHERE)

Сложные логические выражения

В WHERE можно комбинировать условия:

- AND, OR
- NOT
- BETWEEN a AND b вместо двойного сравнения
- IN (...) / NOT IN (...) для множеств значений
- LIKE / NOT LIKE для шаблонов строк (% в PostgreSQL/MySQL, * в Access).

Пример (диапазон + несколько значений):

```
SELECT name
```

```
FROM product
```

```
WHERE price BETWEEN 100 AND 500
```

```
AND category_id IN (1, 3, 5);
```

Агрегатные функции, GROUP BY и HAVING

Основные агрегатные функции

- COUNT(*) – количество строк
- SUM(col) – сумма
- AVG(col) – среднее
- MIN(col) / MAX(col) – минимум / максимум

Особенности:

- Все, кроме COUNT(*), игнорируют NULL.

GROUP BY

Если в SELECT есть и обычные столбцы, и агрегатные функции, то все **неагрегатные** столбцы должны быть перечислены в GROUP BY.

```
SELECT department, COUNT(*) AS total_employees
```

```
FROM employees
```

```
GROUP BY department;
```

HAVING

HAVING фильтрует **уже сгруппированные** данные, то есть условия по результатам агрегатных функций:

```
SELECT department, COUNT(*) AS total_employees
```

```
FROM employees
```

```
GROUP BY department
```

```
HAVING COUNT(*) >= 5;
```

Многотабличные запросы и JOIN

- INNER JOIN
Берет только строки, где есть совпадающие ключи в обеих таблицах.
- LEFT JOIN
Все строки из левой таблицы + подходящие из правой (иначе NULL).
- RIGHT JOIN
Аналогично, только наоборот.
- Декартово произведение (просто перечисление таблиц через запятую в FROM)
В реальных задачах почти всегда комбинируется с WHERE, чтобы не получить полный «кросс-продукт».

Пример:

```
SELECT p.name_product, c.name_category
```

```
FROM product p
```

```
INNER JOIN category c ON p.category_id = c.category_id;
```

Для удаления дублей используют DISTINCT:

```
SELECT DISTINCT c.name_category  
FROM product p  
JOIN category c ON p.category_id = c.category_id;
```

Вложенные подзапросы

Общая идея

Подзапрос (nested / subquery) — запрос SELECT внутри другого запроса. Используется, когда условие зависит от результата другого запроса.

Подзапрос может вернуть:

- одно значение (скалярный);
- несколько значений в одном столбце;
- целую таблицу (несколько столбцов и строк).

Подзапросы в WHERE

Пример: выбрать животных, у которых продолжительность жизни в два раза меньше средней:

```
SELECT name  
FROM animal  
WHERE life_time = (  
    SELECT AVG(life_time) / 2  
    FROM animal  
);
```

- Если используем обычные операторы ($=$, $<$, $>$...), подзапрос должен вернуть **одно** значение.
- Если подзапрос может вернуть много значений, нужно IN / NOT IN.

```
SELECT name  
FROM animal  
WHERE life_time IN (  
    SELECT life_time / 2  
    FROM animal
```

```
WHERE name IN ('Заяц', 'Медведь', 'Дельфин')
);
```

Подзапросы в FROM (таблица из подзапроса)

Используем подзапрос как виртуальную таблицу:

```
SELECT a.name
FROM animal a
JOIN (
    SELECT *
    FROM class
    WHERE name_class = 'Млекопитающие'
) c ON a.class_id = c.class_id;
```

Подзапросы в SELECT

Можно посчитать агрегат по всей таблице и использовать его как столбец:

```
SELECT zoo.name_zoo,
    SUM(n.count) /
    (SELECT SUM(count) FROM presence) AS ratio
FROM zoo
JOIN presence n ON zoo.id = n.zoo_id
GROUP BY zoo.name_zoo;
```

Коррелированные подзапросы

Коррелированный подзапрос использует столбцы из внешнего запроса, выполняется **для каждой строки** внешнего набора.

Пример идеи (с студентами и их средним баллом по группе):

```
SELECT s.student_id, s.name
FROM student s
WHERE avg_mark(s.student_id) >
    (SELECT AVG(mark)
    FROM student s2
```

WHERE s2.group_id = s.group_id);

Коррелированные подзапросы мощные, но могут быть тяжелыми по производительности.

Операции над наборами: UNION, UNION ALL и др.

Операции теории множеств по данным: UNION, UNION ALL, INTERSECT, EXCEPT / MINUS.

UNION и UNION ALL

- UNION объединяет результаты двух запросов, **удаляя дубликаты**.
- UNION ALL делает то же, но **оставляет дубликаты** и обычно работает быстрее.

Ограничения:

- Число столбцов и типы в обеих частях должны совпадать.
- Итоговая структура наследуется от первого (левого) запроса.

Пример:

(SELECT name, amount FROM sales_2024)

UNION ALL

(SELECT name, amount FROM sales_2025);

Общие табличные выражения (CTE, WITH) как способ организовать сложный запрос

Общее табличное выражение (CTE)

- Это **временный именованный результат запроса**, которым можно пользоваться в пределах одного большого запроса.
- Записывается через WITH перед основным SELECT. Пример:

WITH top_customers AS (

SELECT buyer_id, SUM(total_amount) AS total_sum

FROM orders

GROUP BY buyer_id

HAVING SUM(total_amount) > 100000

)

```
SELECT b.name_buyer, t.total_sum  
FROM top_customers t  
JOIN buyer b ON b.buyer_id = t.buyer_id;
```

Зачем нужно:

- Разбивает большой монстр-запрос на части.
- Упрощает чтение и отладку.
- Иногда помогает оптимизатору.

Источники:

<https://skado.khpi.ru/indicationsvkr/6625/>

<https://practicum.yandex.ru/blog/sql-zaprosy-dlya-upravleniya-bazami-dannyh/>

<https://postgrespro.ru/docs/postgrespro/current/queries-with>

https://ru.wikipedia.org/wiki/Union_%28SQL%29

2. Разработайте сложные запросы к базе данных, созданной и заполненной на предыдущих занятиях.

Вывести список заказов с именем и телефоном покупателя, а также статус заказа. join - заказы + покупатели

```
SELECT  
    order_product.order_id,  
    order_product.date_order,  
    order_product.order_status,  
    buyer.name_buyer,  
    buyer.phone_number  
FROM order_product  
JOIN buyer ON order_product.buyer_id = buyer.buyer_id  
ORDER BY order_product.date_order DESC;
```

	order_id integer 🔒	date_order timestamp without time zone 🔒	order_status text 🔒	name_buyer text 🔒	phone_number character varying (20) 🔒
1	1	2025-10-28 14:12:01.782478	сборка	Иван Петров	+79990001122
2	2	2025-10-28 14:12:01.782478	в пути	Мария Смирнова	+79995554433
3	3	2025-10-28 14:12:01.782478	отменен	Дмитрий Кузнец...	+79998887766
4	4	2025-10-28 14:12:01.782478	в обработке	Анна Попова	+79997776655
5	5	2025-10-28 14:12:01.782478	в пути	Сергей Иванов	+79993332211
6	6	2025-10-28 14:12:01.782478	сборка	Ольга Павлова	+79992225544
7	7	2025-10-28 14:12:01.782478	в обработке	Никита Соколов	+79991114455
8	8	2025-10-28 14:12:01.782478	в пути	Татьяна Морозо...	+79990002233

Выбор товаров дороже среднего AVG + MAX

SELECT

product.product_id,
product.name_product,
product.price

FROM product

WHERE product.price > (SELECT AVG(product.price) FROM product)

ORDER BY product.price DESC;

	product_id [PK] integer ✎	name_product text ✎	price double precision ✎	max_price_all double precision 🔒
1	9	Телевизор	78000	78000
2	3	Смартфон	52000	78000

Показать всех покупателей, которые сделали не менее двух заказов COUNT + GROUP BY + HAVING

SELECT

buyer.buyer_id,
buyer.name_buyer,
COUNT(order_product.order_id) AS orders_count

FROM buyer

JOIN order_product ON buyer.buyer_id = order_product.buyer_id

GROUP BY buyer.buyer_id, buyer.name_buyer

HAVING COUNT(order_product.order_id) >= 1

ORDER BY orders_count DESC;

	buyer_id [PK] integer	name_buyer text	orders_count bigint
1	4	Анна Попова	1
2	6	Ольга Павлова	1
3	2	Мария Смирнова	1
4	3	Дмитрий Кузнец...	1
5	5	Сергей Иванов	1
6	7	Никита Соколов	1
7	1	Иван Петров	1
8	8	Татьяна Морозо...	1

Показать покупателей, у которых есть хотя бы один заказ со статусом "в пути" EXISTS

SELECT

buyer.buyer_id,

buyer.name_buyer,

buyer.phone_number

FROM buyer

WHERE EXISTS (

SELECT *

FROM order_product

WHERE order_product.buyer_id = buyer.buyer_id

AND order_product.order_status = 'в пути'

);

	buyer_id [PK] integer 	name_buyer text 	phone_number character varying (20) 
1	2	Мария Смирнова	+79995554433
2	5	Сергей Иванов	+79993332211
3	8	Татьяна Морозо...	+79990002233